

# Intelligent Dead Reckoning

Android application: status and roadmap  
SIH 2026 | Task 4, Mobile Interface and UI | 4 September 2026

**Where the project stands.** The Android application is complete and recording reliably. Dead reckoning with map-matched heading feedback works and is measured. Two further accuracy features are fully built but deliberately switched off, because both depend on one unresolved defect: while running without GNSS, the integrator recovers about 30% less distance than was actually travelled. That defect has now been diagnosed to a specific cause, and fixing it needs no machine-learning model and no external dependency.

## 1. What runs on the phone

A foreground service records eleven sensor streams and drives four estimators in parallel across three threads. There is exactly one writer thread, so no locking is needed anywhere in the recording path.

| THREAD     | OWNS                                                                 | RATE   |
|------------|----------------------------------------------------------------------|--------|
| imu-logger | Sensor, GNSS and location callbacks; all file writes; the integrator | 200 Hz |
| imu-ml     | Model load and inference (about 40 ms per forward pass)              | 10 Hz  |
| map-match  | Mapsforge tile reads, HMM matcher, road graph                        | 0.5 Hz |

Every session writes **imu.csv**, **gps.csv**, **gnss\_status.csv**, **deadreckon.csv**, **ml.csv**, **mapmatch.csv** and **session.json**. All timestamps share the monotonic elapsed-realtime clock, so they can be aligned exactly; session.json records the mapping to UTC. The map draws three tracks at once, GPS, IMU-only and map-snapped, so the divergence a tunnel would cause is visible live.

## 2. What works today

### Application and interface (Task 4)

Essentially complete. Offline OpenStreetMap rendering, radius preloading, downloadable map zones for other parts of India, theme and settings screens, collapsible session card, and a locate button that zooms to a 1 km radius only when currently showing more. Records reliably on the Samsung S21 FE.

### Dead reckoning with map-matched heading feedback

A strapdown inertial integrator free-runs whenever GNSS is unavailable. An online HMM map matcher snaps the drifting estimate to the road network and feeds the matched road bearing back as a heading observation. Measured across 46 to 57 synthetic outages per duration:

| ESTIMATOR                                | 10 s   | 30 s   | 60 s   |
|------------------------------------------|--------|--------|--------|
| Dead reckoning + heading feedback (CRSE) | 10.2 m | 36.0 m | 87.5 m |

### On-device road topology

The strongest technical result so far. Mapsforge map files store ways clipped per tile with no node identity, so the network has no connectivity and the conventional answer is to route against an OSRM server, which an offline application cannot do. However, coordinates are stored in microdegrees, roughly 0.11 m, so segments that once shared an OpenStreetMap node still come back on identical coordinates. Snapping endpoints onto a

half-metre grid rebuilds the graph entirely on the device.

| MEASURE                                  | RESULT |
|------------------------------------------|--------|
| Segments around IIT Kharagpur            | 25,383 |
| Nodes recovered                          | 18,570 |
| Segments gaining at least one link       | 100.0% |
| Segments in a single connected component | 98.5%  |
| Mean node degree                         | 2.73   |

No server, no additional download, and it works underground. Tile-boundary clipping heals for free, because both halves of a clipped way end on the same border coordinate.

### Evaluation harness

CRSE, CAE and AEPS implemented to the same definitions the model side of the project uses, so results are directly comparable. Synthetic outages are cut from recorded sessions and scored against GPS ground truth.

## 3. Built, tested, and deliberately switched off

Two complete features are one line from being live. Both are disabled because measurement showed they would currently make the application worse, not better.

### Along-road tracking

Walks position as a scalar along a road polyline, so the road supplies direction and heading error stops entering the answer altogether. This is the mechanism behind the accuracy of the server-based approach, reproduced without a server.

| CONFIGURATION                                | 10 s   | 30 s   | 60 s    |
|----------------------------------------------|--------|--------|---------|
| Dead reckoning + heading feedback (baseline) | 10.2 m | 36.0 m | 87.5 m  |
| Along-road, fed the integrator's distance    | 12.6 m | 40.8 m | 104.7 m |
| Along-road, fed true distance                | 8.3 m  | 22.3 m | 50.5 m  |

Given a trustworthy distance it cuts 60-second outage error by **42%**. Given the distance we can currently supply, it is 20% worse than doing nothing. The idea and the implementation are both sound; the input is not.

Two findings from building it are worth recording. Seeding matters: entering the road at the moment GNSS is lost, when position is still exact, is 42% better, whereas entering later from a drifted position is 20% worse. Same code, different starting point. And an earlier encouraging result was withdrawn after it failed to survive a denser sample; it had been measured on five outages.

### Model speed fusion

The integrator can now accept the neural network's speed estimate, blended into velocity magnitude only and weighted by a scalar Kalman gain formed from the model's own reported variance. A confident stand-still prediction sets the target speed to zero. It is applied only while unaided, since GNSS velocity beats anything a phone-IMU network can offer.

Retrained on IO-VNBD, the model beats a constant for the first time. Against the held-out test runs it reaches 4.88 m/s speed RMSE where predicting a constant gives 6.84, and correlation with truth is +0.74. The previous checkpoint was twice WORSE than a constant and anti-correlated at -0.22, and its yaw head had been regressing on a hardcoded zero.

It is still not accurate enough to steer the integrator, and the reason has been diagnosed. The model overfits by a factor of twelve:

| SPLIT      | SPEED RMSE | CORRELATION | SD(PRED)/SD(TRUTH) |
|------------|------------|-------------|--------------------|
| train      | 0.396 m/s  | 0.999       | 0.968              |
| validation | 2.796 m/s  | 0.865       | 0.913              |
| test       | 4.877 m/s  | 0.739       | 0.629              |

A train correlation of 0.999 means it has memorised the training windows: 3,848,196 parameters against 8,804 of them. On an unseen session it falls back toward the training mean, which is why its per-run bias correlates -0.82 with that run's mean speed and why the spread of its predictions collapses to 0.63 of the truth's. Two other explanations were tested and rejected - the estimated mounting rotation (correlation with bias only +0.28) and irreducible observability (ruled out by the near-perfect training fit). The levers are capacity, regularisation and data.

## 4. The blocking defect, and its cause

**While free-running without GNSS, the integrator recovers about 30% less distance than was actually travelled.** This is the single largest error source in the system, and it is what keeps both features above switched off.

| OUTAGE DURATION                    | 10 s  | 30 s  | 60 s  |
|------------------------------------|-------|-------|-------|
| Distance shortfall                 | 15.0% | 21.3% | 29.5% |
| Zero-velocity updates firing       | 1.9%  | 0.0%  | 1.4%  |
| Samples dropped by the timing gate | 0.0%  | 0.0%  | 0.0%  |

### What was ruled out

- **The zero-velocity update is not the cause.** It was the leading hypothesis, and it is wrong. It fires on 0 to 2% of samples during outages, and at 30 seconds the shortfall is 21.3% while it fires on 0.0% of them. Disabling it entirely moves the 60-second shortfall only from 29.5% to 24.7%.
- **Dropped samples are not the cause.** The timing gate discards 0.0%.

### What it actually is

A systematic deceleration. Over 46 sixty-second outages, the levelled acceleration projected onto the direction of travel averages **-0.0195 m/s<sup>2</sup>**, and is negative in **89%** of them. Over 60 seconds that alone removes 1.17 m/s, roughly a third of a 3.4 m/s cruise, which accounts for the entire shortfall. The shortfall also grows with duration exactly as a constant deceleration would.

Two supporting measurements point at where it comes from. The levelled vertical axis reads 9.7574 m/s<sup>2</sup> while the accelerometer magnitude averages 9.8864, so gravity is not aligned with the estimated vertical and leaks into the horizontal plane. And the assumed gravity constant, 9.8066 m/s<sup>2</sup>, is 0.08 m/s<sup>2</sup> away from this device's own mean.

**The structural fault:** the horizontal bias states are learned only during stand-still, so they are estimated exactly when they are not needed and frozen exactly when they are. During a tunnel transit, nothing corrects them.

Whole sessions are unaffected, with a distance ratio of 0.992, because stand-still periods keep re-anchoring the estimate. The shortfall is specific to unanchored free-running, which is precisely the case the application exists for.

A separate and distinct failure was also found: in one near-stationary session, 39.8% of moving samples were wrongly called stationary and the integrator recovered 11 m of 93 m. That is a real defect, but it is not the cause of the general shortfall.

## 5. The physics-informed loss, settled by measurement

The execution plan specifies a penalty enforcing  $v(t) = v(t-1) + a \cdot dt$ . Measured against a held-out test set it HURTS at every weight tried: +1.9% at 0.01, +3.8% at 0.05, and +23.6% at 0.2, where training destabilises outright. The reason is that the integrator already satisfies that identity exactly and in closed form, so penalising violations of it teaches the network nothing the data does not already carry.

A different constraint does slightly better. The centripetal identity, lateral acceleration equals speed times yaw rate, couples two heads through physics using only the input signal, and it holds almost exactly in this data - regressing the vehicle's own lateral accelerometer on that product gives a slope of 1.008 with correlation 0.941. Applied to the phone's own lateral channel, as it would have to be on a real handset, it is worth -1.35%, consistent in sign across four seeds but short of statistical significance. It is not the thing standing between this project and working navigation.

## 6. What to build next

Ordered by value per unit of effort. Item 1 is now well specified by the diagnosis above and depends on nobody else.

| # | WORK                                         | WHY IT MATTERS                                                                                                                                                                                                                                                                                | DEPENDS ON |
|---|----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| 1 | <b>Gyro and accelerometer bias observers</b> | Learn both biases while GNSS is healthy and carry them into the outage. Measured: replacing the model's yaw head with the raw gyro and removing its bias from pre-outage data alone takes 300 s free-running drift from 46.1% of distance travelled to 24.0%. Needs no model and no new data. | Nothing    |
| 2 | <b>Cut model capacity and regularise</b>     | The model overfits twelvefold. Two of the reviewed papers independently recommend shrinking the network; the constructor now spans 3.85M down to 28,580 parameters, with dropout and weight decay exposed.                                                                                    | Nothing    |
| 3 | <b>Per-session output calibration</b>        | Fit the model against GNSS over the two minutes before an outage and apply an offset during it. Measured worth about 20% at 30-120 s. Fitting a gain as well overfits and is worse.                                                                                                           | Item 1     |
| 4 | <b>Record driving data at 200 Hz</b>         | IO-VNBD's phone stream is 10 Hz, so its Nyquist is 5 Hz and the vibration band that encodes absolute speed is not sampled at all. Our own app already records the accelerometer at 200 Hz, which no public dataset here can validate.                                                         | Field time |
| 5 | <b>Along-road tracking</b>                   | Even the best configuration still drifts 19-24% at 300 s, and that residual is largely cross-track, which is what the road network removes. The topology and the tracker are already built and gated off.                                                                                     | Items 1-3  |
| 6 | <b>Road-test on the phone</b>                | None of the above has run on the handset with real motion. Everything reported here is measured in the Python harness.                                                                                                                                                                        | Field time |

The verification commands already exist and both currently fail closed, which is the intended behaviour:

| COMMAND                                                       | CHECKS                             |
|---------------------------------------------------------------|------------------------------------|
| <code>python -m eval.dr_diagnostics &lt;sessions&gt;</code>   | Where the distance shortfall goes  |
| <code>python -m eval.model_speed_eval &lt;sessions&gt;</code> | Whether the model beats a constant |
| <code>python -m eval.outage_eval &lt;sessions&gt;</code>      | End-to-end outage position error   |

## 7. Notes for the team

- **The on-device road matcher is new capability worth sharing.** It solves offline, on the handset, what the model-side pipeline currently uses an OSRM server for. Road topology does not require a routing server.
- **The training splits are not reproducible.** The data preparation directory on the model side is not committed, so results cannot be independently checked.
- **CORRECTION: IO-VNBD does contain phone IMU data, and is now the training set.** An earlier version of this document said it could not validate this integrator. That was wrong. The dataset README states the sensors include the inertial sensors and GPS receiver of an Android phone at 10 Hz, about 58 hours over 4,400 km. The copy that looked unusable was a set of Git LFS pointer stubs; the real archives were alongside it.
- **The phone stream is paired with a 10 Hz vehicle reference** carrying position, speed, heading and a real yaw rate. After per-run time alignment and quality gating, 26 runs and 19.9 hours are usable, giving 14,323 training windows against the 304 the previous pipeline produced.
- **Reported numbers come from three walking-pace sessions.** Every outage in them falls below the 5.0 m/s start-speed gate used in the reference literature, so these results are comparable in definition but not directly in magnitude. This is the strongest argument for item 2 above.

All measurements in this document are reproducible from the committed evaluation harness against the recorded sessions. Figures were produced on 4 September 2026.